MATH 345 Skeleton Notes
Units 1–3
Neural networks and attention

© 2024–2026 Jacob Denson, Kaitlyn Phillipson, Sebastien Roch. Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Part I: Neural networks

Unit 1: Layers and affine maps

A neural network is a function built from layers. A layer takes numbers as input, combines them using weights, applies a fixed nonlinear function, called an activation, and sends output numbers to the next layer. We treat the input and output of a layer as vectors. The weights are adjusted from data during training.

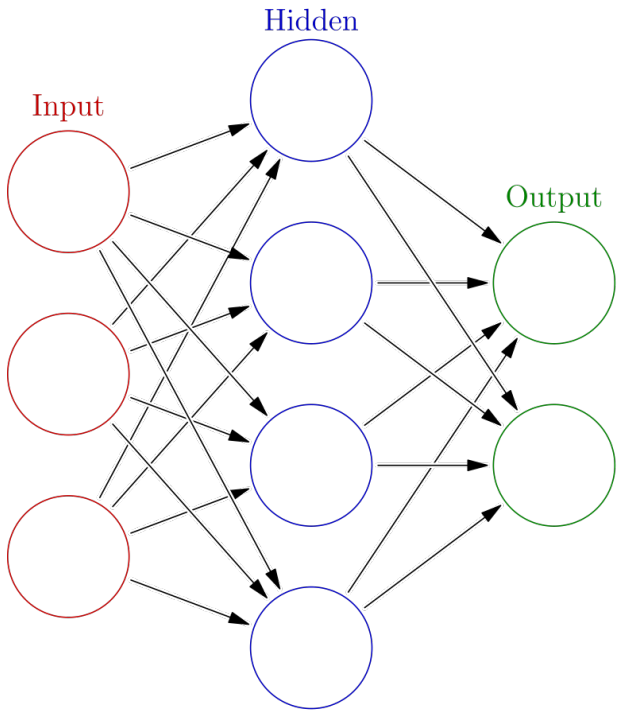


Figure. A neural network diagram with input, hidden, and output layers. Source: Wikipedia, Neural network (machine learning), using Colored neural network.svg by Glosser.ca, derivative of Artificial neural network.svg, licensed under CC BY-SA 3.0.

Example: An affine layer

An affine layer in a neural network takes the form

$$\mathbf{x} \mapsto W\mathbf{x} + \mathbf{b}.$$

The symbol $\mapsto$ means "maps to": the input vector $\mathbf{x}$ is sent to the output vector $W\mathbf{x} + \mathbf{b}$.
If the layer has n inputs and m neurons, then $W \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$.
Writing $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, the coordinate for neuron i is

$$z_i = \sum_{j=1}^{n} W_{ij} x_j + b_i, \qquad i = 1, \dots, m.$$

This is the neuron's weighted sum of the inputs plus its bias. Row i of W contains that neuron's weights. The vector $\mathbf{z}$ collects these quantities for all neurons in the layer.
The matrix W mixes the input coordinates, and the vector $\mathbf{b}$ shifts the result. This map is affine.

Unit 2: Output scores and hidden directions

Activity: Output scores and a hidden direction (U2-LO6)

A final model layer can convert a hidden vector into output scores by

$$\ell = W\mathbf{h} + \mathbf{b}.$$

Let

$$W = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}, \qquad \mathbf{h} = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \qquad \mathbf{d} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}.$$

1. Compute $W\mathbf{h} + \mathbf{b}$.
2. Compute $W\mathbf{d}$.
3. Use part 2 to compute $W(\mathbf{h} + \mathbf{d}) + \mathbf{b}$.
4. Are $\mathbf{h}$ and $\mathbf{h} + \mathbf{d}$ the same hidden vector?
5. Are their output-score vectors the same?
6. Did the bias vector $\mathbf{b}$ affect which hidden-vector change was invisible?



Unit 3: Sigmoid and the chain rule

Recall the affine layer $\mathbf{x} \mapsto W\mathbf{x} + \mathbf{b}$ from Example An affine layer. Each output coordinate is one neuron's weighted sum of the inputs plus its bias. A small neural-network block extends this construction by composing three steps: an affine map, a coordinatewise nonlinear rule, and another affine map.

Note: Sigmoid and its derivative

The sigmoid function is defined by

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

We use σ as an activation function, applied to each neuron's weighted sum of the inputs plus its bias to produce that neuron's output. When σ is applied to a vector, it applies this same scalar function separately to each coordinate.
Writing $\sigma(t) = (1 + e^{-t})^{-1}$, the chain rule gives

$$\begin{aligned}
\sigma'(t) &= -(1 + e^{-t})^{-2}(-e^{-t}) \\
&= \frac{e^{-t}}{(1 + e^{-t})^2} \\
&= \frac{1}{1 + e^{-t}} \left(\frac{(1 + e^{-t}) - 1}{1 + e^{-t}} \right) \\
&= \frac{1}{1 + e^{-t}} \left(1 - \frac{1}{1 + e^{-t}} \right) \\
&= \sigma(t)(1 - \sigma(t)).
\end{aligned}$$

Activity: A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7)

Let

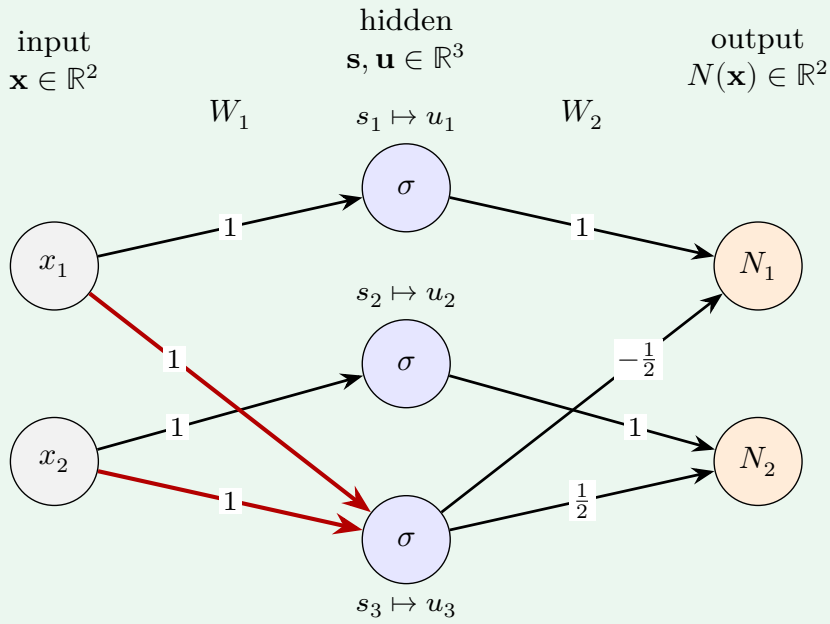
$$W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} 1 & 0 & -1/2 \\ 0 & 1 & 1/2 \end{bmatrix},$$

and let $\mathbf{b}_1 = \mathbf{0}$ and $\mathbf{b}_2 = \mathbf{0}$.
Define

$$\mathbf{s} = W_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{u} = \sigma(\mathbf{s}), \qquad N(\mathbf{x}) = W_2\mathbf{u} + \mathbf{b}_2,$$

where σ is applied coordinatewise. For each neuron $i = 1, 2, 3$ in the first layer, s_i is its weighted sum plus bias, and $u_i = \sigma(s_i)$ is its output after applying the sigmoid function. The second affine map combines these three outputs.

Activity: A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7) (continued)



red edges: weights into neuron 3 = row 3 of W_1, $[1 \quad 1]$

a missing edge is a zero entry of W_1 or W_2

data flows left to right

local matrix: W_1 $D_{\mathbf{a}}$ W_2

$$J_N(\mathbf{a}) = W_2\, D_{\mathbf{a}}\, W_1 \quad \text{(the matrices multiply right to left)}$$

At the base point

$$\mathbf{a} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

do the following.

1. Compute the pre-activation vector $\mathbf{s_a} = W_1\mathbf{a} + \mathbf{b}_1$.
2. Compute the hidden vector $\mathbf{u_a} = \sigma(\mathbf{s_a})$.
3. Compute $N(\mathbf{a})$.
4. Use $\sigma'(0) = 1/4$ to compute

$$D_{\mathbf{a}} = \operatorname{diag}\big(\sigma'((\mathbf{s_a})_1), \sigma'((\mathbf{s_a})_2), \sigma'((\mathbf{s_a})_3)\big).$$

Activity: A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7) (continued)

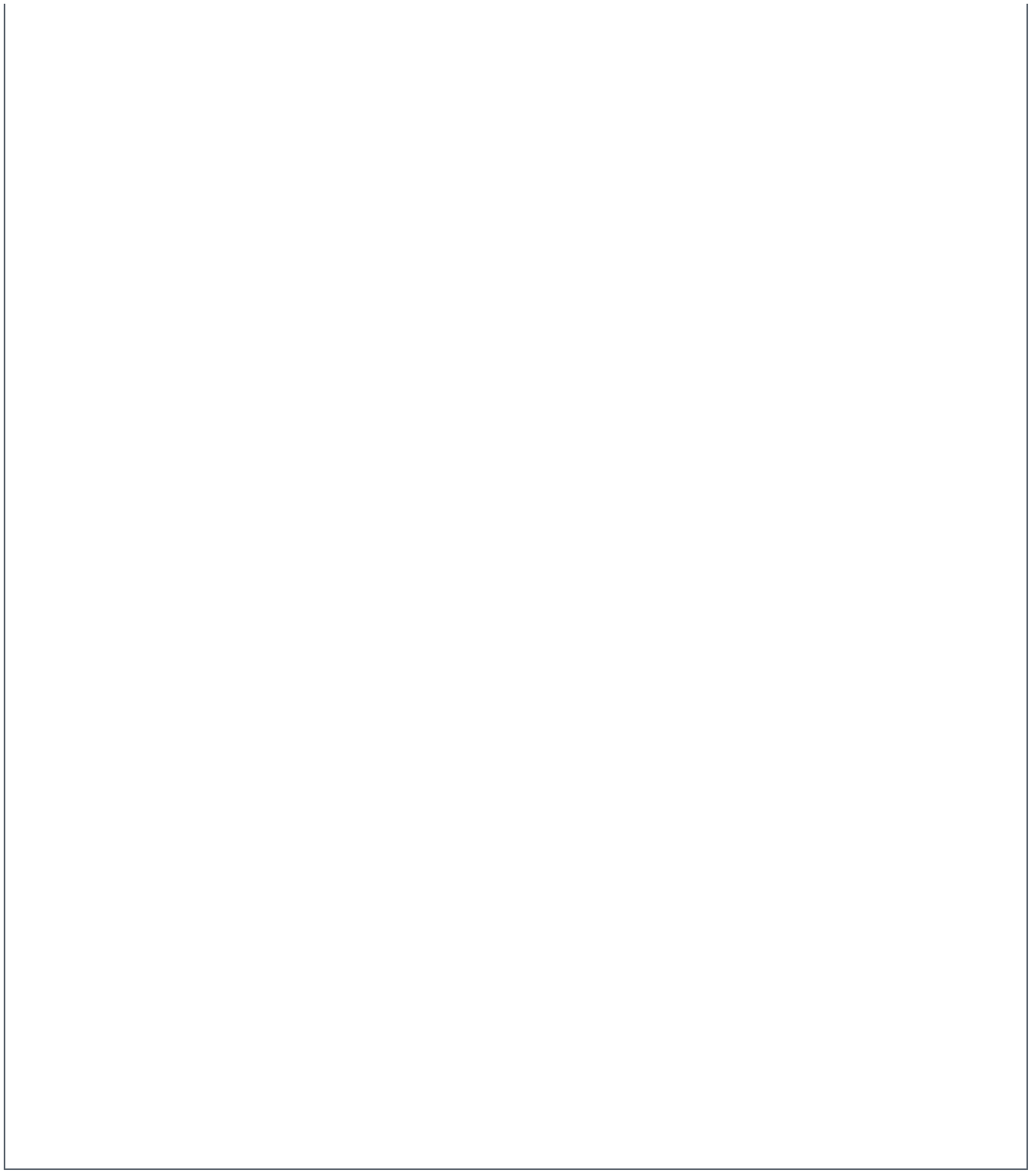
5. Use the chain rule to justify and compute

$$J_N(\mathbf{a}) = W_2 D_{\mathbf{a}} W_1.$$

6. Use

$$N(\mathbf{a} + \mathbf{h}) \approx N(\mathbf{a}) + J_N(\mathbf{a})\mathbf{h}$$

to predict the output change caused by a small input change $\mathbf{h}$.



Note: Why sigmoid here

The sigmoid function is smooth, so the Jacobian matrix of this block is defined at every input. Some neural-network models use nonsmooth activation rules. Those require extra case analysis. Here the point is the chain rule for Jacobian matrices.

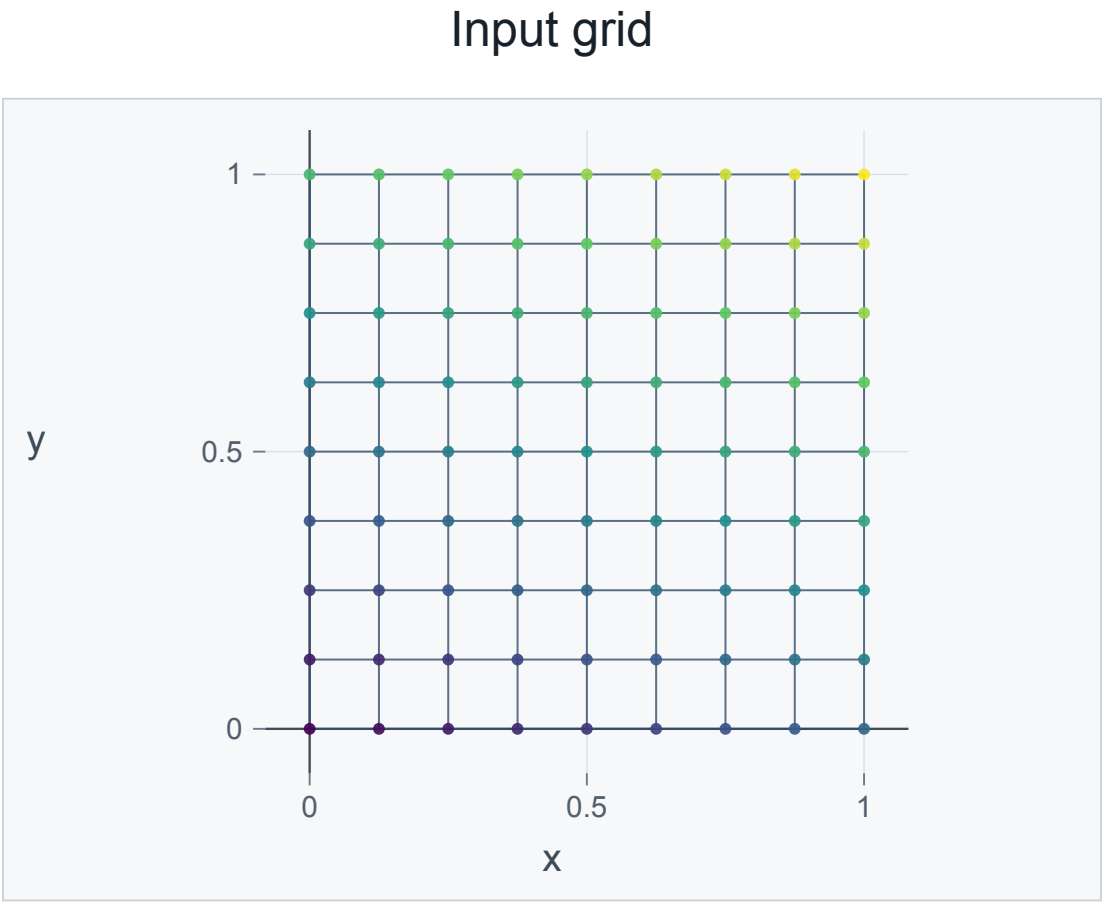


Figure. Input grid. The corresponding output grid follows on the next page.

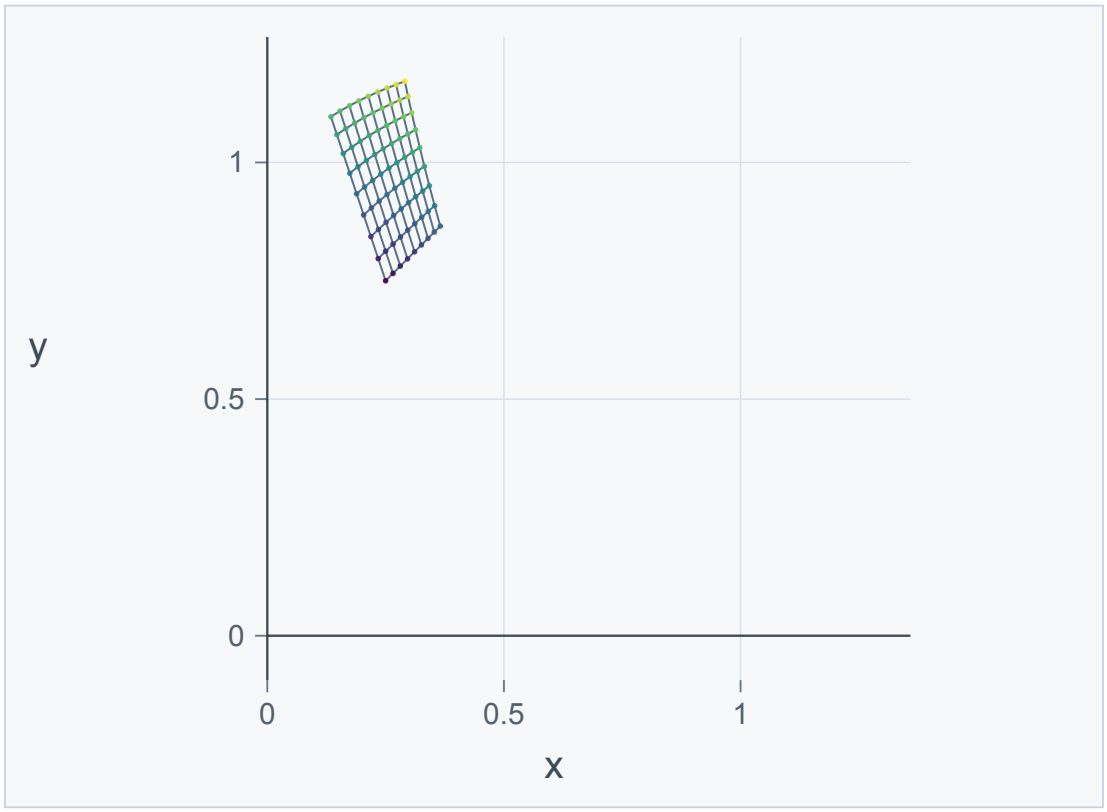


Figure. Square-grid visualization for a two-output sigmoid block.

Part II: Transformers and attention

Unit 1: Transformers and token vectors

A transformer is a neural-network architecture for sequences. In a language model, text is first broken into tokens. A token can be a word, part of a word, punctuation mark, or other text fragment. Each token position carries a vector.

A transformer layer produces a new vector at each token position. The token itself does not change; its vector becomes context-dependent. In the phrase “small red bird”, the new vector at the bird position can gather information from “small” and “red”. In a next-token model, the vector at this final position can then help predict what comes next.

For one output position, its query is used to decide which positions are relevant. Each token has a key used in that comparison and a value containing the information it can contribute. Queries and keys determine the weights; values are what get averaged. Every position has all three roles, but the next activity computes only the attention output at the bird position.

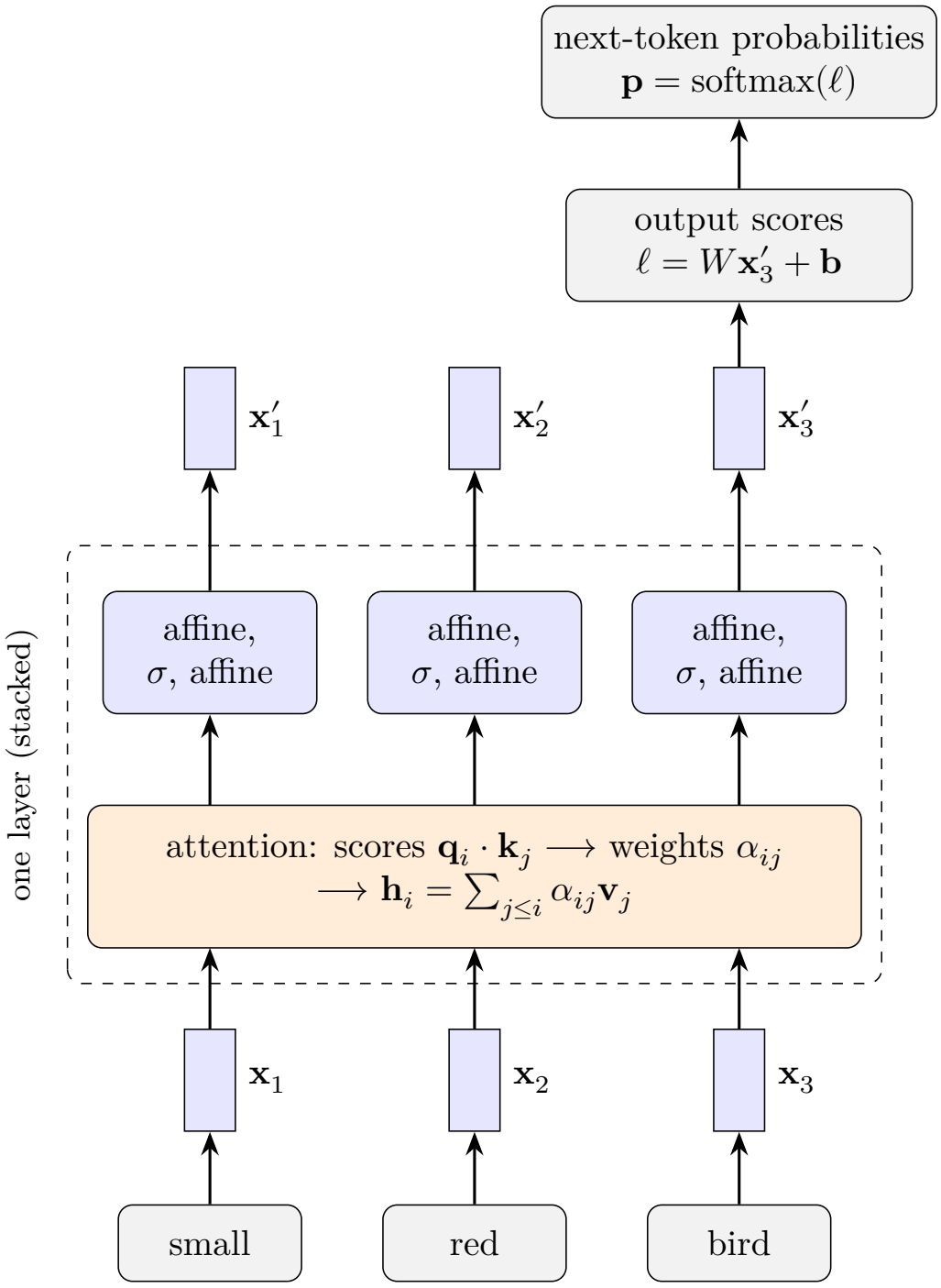


Figure. A simplified next-token transformer, read from bottom to top.

Unit 1: Attention at one destination

Activity: Gathering context for the bird position (U1-LO1, U1-LO8)

Consider the token sequence

$$\text{small}, \qquad \text{red}, \qquad \text{bird}.$$

We compute the attention output at the position occupied by "bird". The bird position is the destination; "small", "red", and "bird" itself are possible sources of information.
The query $\mathbf{q}_{\text{bird}}$ is compared with each key $\mathbf{k}_i$ to produce a relevance score. The scores are converted into weights, and those weights are used to average the value vectors:

$$\begin{aligned}\text{query-key scores} &\longrightarrow \text{weights} \\ &\longrightarrow \text{weighted average of values.}\end{aligned}$$

For this activity, the score for token i is

$$\text{score}_i = \mathbf{q}_{\text{bird}} \cdot \mathbf{k}_i.$$

Use

$$\mathbf{q}_{\text{bird}} = \begin{bmatrix}1\\1\end{bmatrix}, \qquad \mathbf{k}_{\text{small}} = \begin{bmatrix}1\\0\end{bmatrix}, \qquad \mathbf{k}_{\text{red}} = \begin{bmatrix}0\\1\end{bmatrix}, \qquad \mathbf{k}_{\text{bird}} = \begin{bmatrix}1\\1\end{bmatrix}.$$

1. Compute the three scores

$$\mathbf{q}_{\text{bird}} \cdot \mathbf{k}_{\text{small}}, \qquad \mathbf{q}_{\text{bird}} \cdot \mathbf{k}_{\text{red}}, \qquad \mathbf{q}_{\text{bird}} \cdot \mathbf{k}_{\text{bird}}.$$

2. Normalize the positive scores by dividing each score by the sum of all three scores. Call the resulting weights

Activity: Gathering context for the bird position (U1-LO1, U1-LO8) (continued)

$$\alpha = \begin{bmatrix} \alpha_{\text{small}} \\ \alpha_{\text{red}} \\ \alpha_{\text{bird}} \end{bmatrix}.$$

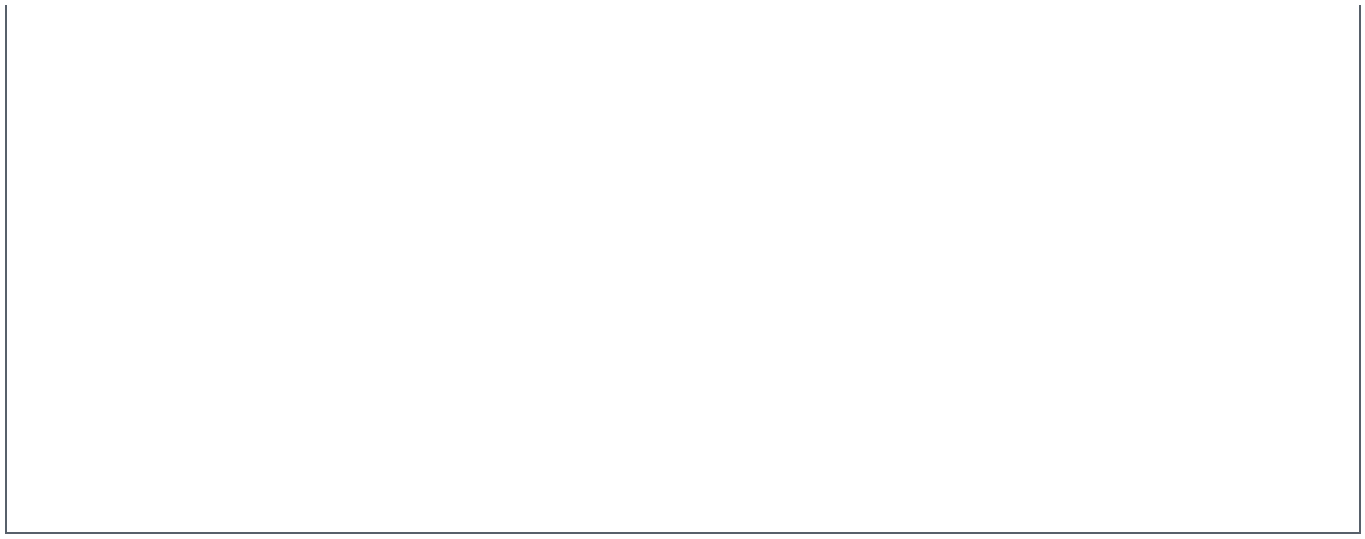
3. The coordinates of the value vectors are abstract toy features. Use

$$\mathbf{v}_{\text{small}} = \begin{bmatrix} 4 \\ 0 \end{bmatrix}, \qquad \mathbf{v}_{\text{red}} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}, \qquad \mathbf{v}_{\text{bird}} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

   to compute

$$\mathbf{h}_{\text{bird}} = \alpha_{\text{small}}\mathbf{v}_{\text{small}} + \alpha_{\text{red}}\mathbf{v}_{\text{red}} + \alpha_{\text{bird}}\mathbf{v}_{\text{bird}}.$$

4. Which token receives the largest weight? What total weight is assigned to “small” and “red” together? What does this say about the information gathered at the bird position?



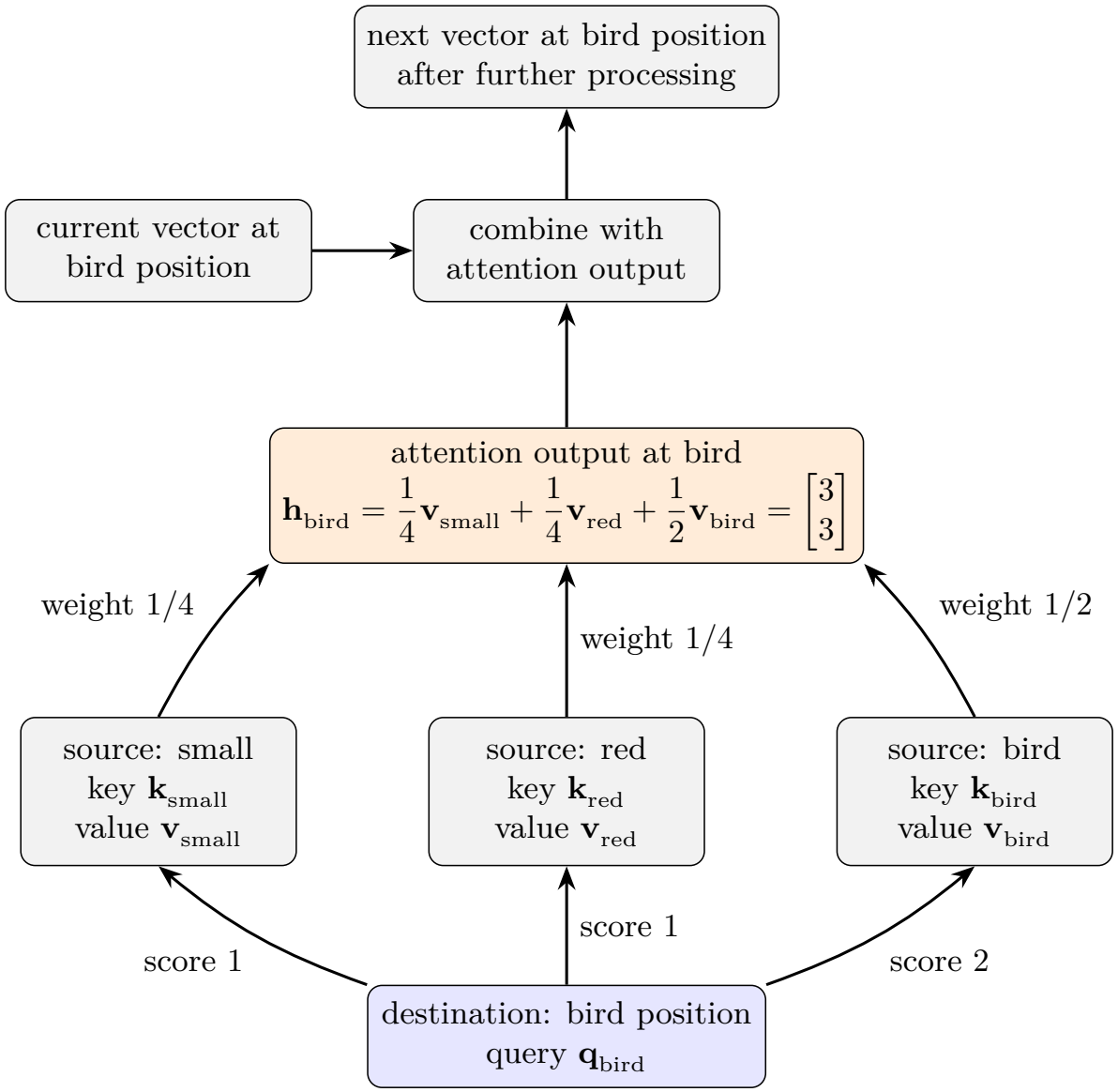


Figure. Attention at the bird position combines value vectors from small, red, and bird.

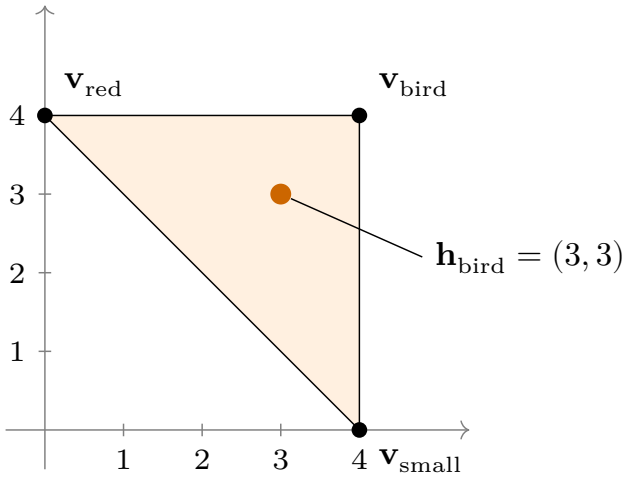


Figure. The attention output at the bird position lies in the triangle of value vectors.

Warning

Gathering context for the bird position (U1-LO1, U1-LO8) isolates one part of a transformer layer. It uses normalization by the sum of positive scores to keep the arithmetic simple. Real transformer attention usually uses a different rule to convert scores into weights, but it follows the same pattern:

$$\text{scores} \longrightarrow \text{weights} \longrightarrow \text{weighted average of values.}$$

A full layer then combines the attention output with the current token representation and processes the result further.

Example: The attention activity in matrix form

In Gathering context for the bird position (U1-LO1, U1-LO8), we computed the scores

$$\mathbf{q}_{\text{bird}} \cdot \mathbf{k}_{\text{small}}, \qquad \mathbf{q}_{\text{bird}} \cdot \mathbf{k}_{\text{red}}, \qquad \mathbf{q}_{\text{bird}} \cdot \mathbf{k}_{\text{bird}}$$

one at a time. Matrix-vector multiplication computes the same scores at once.

Put the key vectors as the rows of

$$K = \begin{bmatrix} \mathbf{k}_{\text{small}}^T \\ \mathbf{k}_{\text{red}}^T \\ \mathbf{k}_{\text{bird}}^T \end{bmatrix}.$$

The matrix K collects the already-computed key vectors. Later, we will see how a learned weight matrix W_K produces these vectors from the token representations. Then

$$K\mathbf{q}_{\text{bird}} = \begin{bmatrix} \mathbf{k}_{\text{small}} \cdot \mathbf{q}_{\text{bird}} \\ \mathbf{k}_{\text{red}} \cdot \mathbf{q}_{\text{bird}} \\ \mathbf{k}_{\text{bird}} \cdot \mathbf{q}_{\text{bird}} \end{bmatrix}.$$

For the numbers in Gathering context for the bird position (U1-LO1, U1-LO8),

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{q}_{\text{bird}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

so

$$K\mathbf{q}_{\text{bird}} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}.$$

Next, divide each positive score by their sum, $1 + 1 + 2 = 4$, to obtain the attention weights

$$\alpha = \begin{bmatrix} \alpha_{\text{small}} \\ \alpha_{\text{red}} \\ \alpha_{\text{bird}} \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1/4 \\ 1/4 \\ 1/2 \end{bmatrix}.$$

Put the value vectors from Gathering context for the bird position (U1-LO1, U1-LO8) in the rows of

$$V = \begin{bmatrix} \mathbf{v}_{\text{small}}^T \\ \mathbf{v}_{\text{red}}^T \\ \mathbf{v}_{\text{bird}}^T \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 4 \\ 4 & 4 \end{bmatrix}.$$

Multiplying V on the left by the row vector α^T forms the weighted average of its rows:

$$\begin{aligned} \mathbf{h}_{\text{bird}}^T &= \alpha^T V \\ &= \alpha_{\text{small}} \mathbf{v}_{\text{small}}^T + \alpha_{\text{red}} \mathbf{v}_{\text{red}}^T + \alpha_{\text{bird}} \mathbf{v}_{\text{bird}}^T \\ &= \begin{bmatrix} 1/4 & 1/4 & 1/2 \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 4 \\ 4 & 4 \end{bmatrix} \\ &= \begin{bmatrix} 3 & 3 \end{bmatrix}. \end{aligned}$$

Equivalently,

$$\mathbf{h}_{\text{bird}} = V^T \alpha = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

This is the attention output at the bird position.

Activity: The same attention calculation in code (U1-LO4, U1-LO8)

The following code performs the calculation from the token attention activity.

```
import numpy as np

tokens = np.array(["small", "red", "bird"])

q = np.array([1., 1.])

K = np.array([
    [1., 0.],
    [0., 1.],
    [1., 1.],
])

V = np.array([
    [4., 0.],
    [0., 4.],
    [4., 4.],
])

s = K @ q
alpha = s / s.sum()
out = alpha @ V

s, alpha, out
```

Output:

```
(array([1., 1., 2.]),
 array([0.25, 0.25, 0.5 ]),
 array([3., 3.]))
```

1. Which line computes the token scores?

Activity: The same attention calculation in code (U1-LO4, U1-LO8) (continued)

2. Which token receives the largest weight?
3. Which line forms the weighted average?
4. Why do the entries of `alpha` add to 1?

Unit 1: Many token queries and keys

Example: Many token queries at once

Suppose a sequence has L tokens. Store the token vectors as rows of a matrix

$$X = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_L^T \end{bmatrix} \in \mathbb{R}^{L \times d}.$$

A transformer layer forms three views of the same token vectors:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

The matrices W_Q, W_K, W_V are weight matrices. Their entries are adjusted during training. For now, treat them as ordinary matrices that assign each token three roles.

For destination position i, the vector $\mathbf{q}_i$ is its query: what that position uses to decide which source positions are relevant to its attention output.

For source position j, the vector $\mathbf{k}_j$ is its key: what is compared with the query from a destination position.

For source position j, the vector $\mathbf{v}_j$ is its value: the information that position can contribute after weights are chosen.

Write

$$Q = \begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_L^T \end{bmatrix}, \qquad K = \begin{bmatrix} \mathbf{k}_1^T \\ \mathbf{k}_2^T \\ \vdots \\ \mathbf{k}_L^T \end{bmatrix}.$$

Then

$$K^T = \begin{bmatrix} \mathbf{k}_1 & \mathbf{k}_2 & \cdots & \mathbf{k}_L \end{bmatrix}.$$

So

$$QK^T = \begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_L^T \end{bmatrix} \begin{bmatrix} \mathbf{k}_1 & \mathbf{k}_2 & \cdots & \mathbf{k}_L \end{bmatrix} = \begin{bmatrix} \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \cdots & \mathbf{q}_1 \cdot \mathbf{k}_L \\ \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \cdots & \mathbf{q}_2 \cdot \mathbf{k}_L \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{q}_L \cdot \mathbf{k}_1 & \mathbf{q}_L \cdot \mathbf{k}_2 & \cdots & \mathbf{q}_L \cdot \mathbf{k}_L \end{bmatrix}.$$

Thus the score matrix

$$S = QK^T$$

has entries

$$S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j.$$

Row i of S contains the scores used when computing the attention output at position i.

Activity: Many-token shape and entry check (U1-LO3, U1-LO6)

Suppose

$$X \in \mathbb{R}^{5\times 4}$$

and

$$W_Q, W_K, W_V \in \mathbb{R}^{4\times 3}.$$

Let

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad S = QK^T.$$

1. What are the shapes of Q, K, and V?
2. What is the shape of K^T?
3. What is the shape of $S = QK^T$?

Activity: Many-token shape and entry check (U1-LO3, U1-LO6) (continued)

4. Suppose

$$\mathbf{q}_2 = \begin{bmatrix} q_{21} \\ q_{22} \\ q_{23} \end{bmatrix}, \qquad \mathbf{k}_4 = \begin{bmatrix} k_{41} \\ k_{42} \\ k_{43} \end{bmatrix}.$$

   Expand the entry S_{24}.

5. What does S_{24} compare?

6. What does row 2 of S contain?

From attention scores to attention output

An attention calculation has three steps:

dot-product scores $\longrightarrow$ weights $\longrightarrow$ weighted averages.

The score S_{ij} compares the query at destination position i with the key at source position j. A larger score means source position j is more relevant to the attention output at destination position i.
The scores are not yet the output. For each token i, a weighting rule converts row i of S into numbers

$$\alpha_{i1}, \alpha_{i2}, \ldots, \alpha_{iL}$$

that are nonnegative and add to 1. These are the attention weights for token i. They say how much token i uses each value vector.
Let

$$A_{\text{att}} = [\alpha_{ij}]$$

be the attention-weight matrix. If the value vectors are stored as rows,

$$V = \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_L^T \end{bmatrix},$$

then the attention output is

$$H = A_{\text{att}} V.$$

Row i of H is

$$\mathbf{h}_i^T = \alpha_{i1}\mathbf{v}_1^T + \alpha_{i2}\mathbf{v}_2^T + \cdots + \alpha_{iL}\mathbf{v}_L^T.$$

Equivalently,

$$\mathbf{h}_i = \alpha_{i1}\mathbf{v}_1 + \alpha_{i2}\mathbf{v}_2 + \cdots + \alpha_{iL}\mathbf{v}_L.$$

Thus H is the attention-output matrix, and each row of H is a weighted average of value vectors. These rows contribute to later token representations; a full transformer layer combines them with the current representations and processes them further.

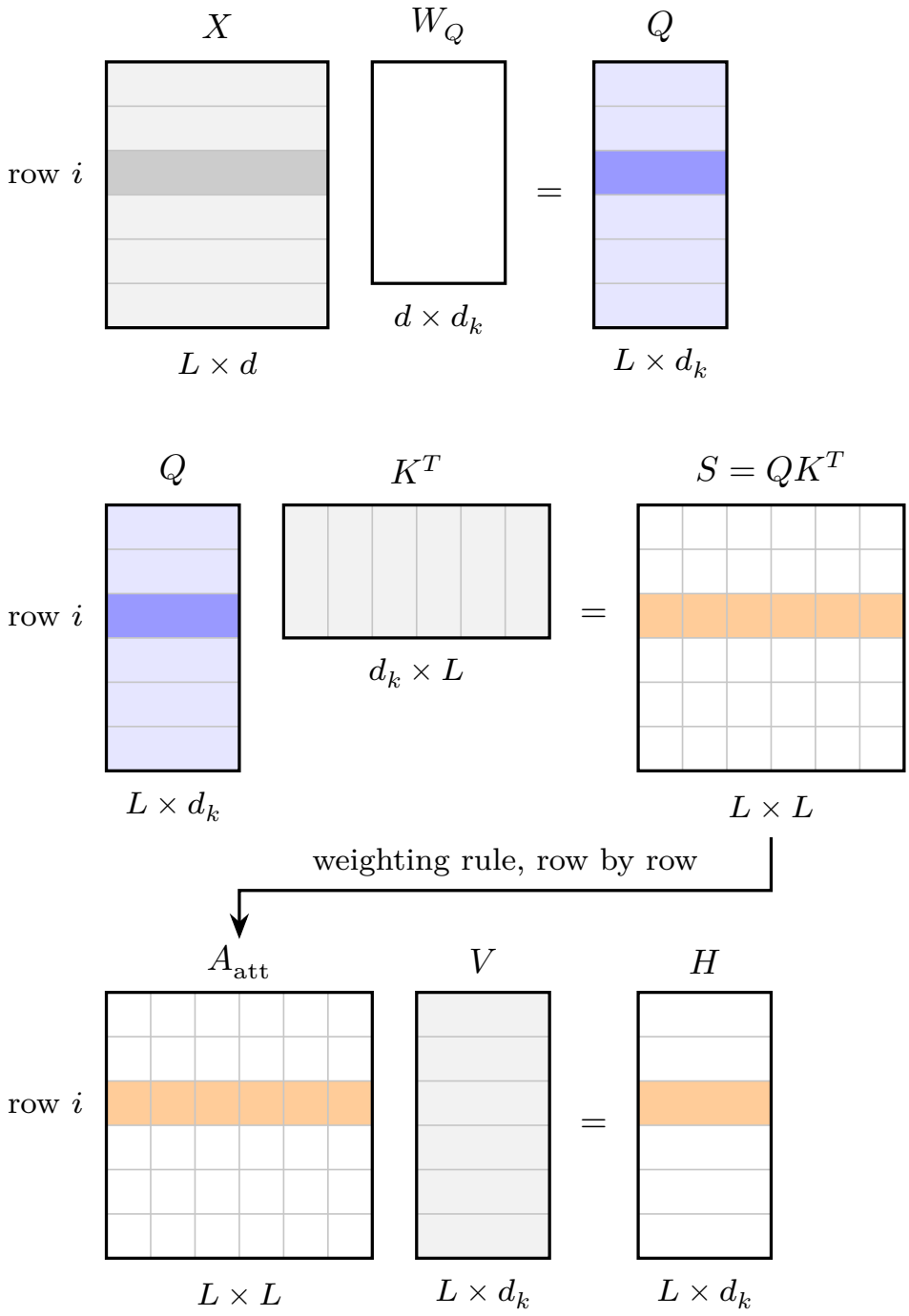


Figure. The shapes of the matrices in the attention calculation, with one destination row highlighted.

Activity: Many-token attention output (U1-LO3, U1-LO6, U1-LO8)

Suppose

$$X \in \mathbb{R}^{4\times 3}, \qquad W_Q, W_K, W_V \in \mathbb{R}^{3\times 2}.$$

Define

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \qquad S = QK^T.$$

A weighting step converts S into an attention-weight matrix

$$A_{\text{att}} \in \mathbb{R}^{4\times 4}.$$

The attention output is

$$H = A_{\text{att}}V.$$

1. What are the shapes of Q, K, and V?
2. What is the shape of $S = QK^T$?
3. What does S_{ij} compare?
4. What does row i of A_{att} tell you?
5. What is the shape of H?
6. Why is each row of H a weighted average of value vectors?
7. Now suppose row 2 of A_{att} is

$$\alpha_2^T = \begin{bmatrix} 0 & 1/2 & 1/4 & 1/4 \end{bmatrix}$$

and

Activity: Many-token attention output (U1-LO3, U1-LO6, U1-LO8) (continued)

$$V = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 0 & 4 \\ 4 & 4 \end{bmatrix}.$$

Compute the attention-output vector $\mathbf{h}_2$.

Activity: The same shape check in code (U1-LO3, U1-LO8)

The following code performs a many-token shape check similar to the previous activity.

```
import numpy as np

X_tokens = np.ones((5, 4))
WQ = np.ones((4, 3))
WK = np.ones((4, 3))
WV = np.ones((4, 3))

Q = X_tokens @ WQ
K_tokens = X_tokens @ WK
V_tokens = X_tokens @ WV
scores_many = Q @ K_tokens.T

Q.shape, K_tokens.shape, V_tokens.shape, scores_many.shape
```

Output:

```
((5, 3), (5, 3), (5, 3), (5, 5))
```

1. Which line creates all query-key scores at once?
2. Why is `scores_many.shape` equal to `(5, 5)`?

Warning

In real transformer attention, scores are usually scaled by dividing by $\sqrt{d_k}$, where d_k is the key-vector dimension. In next-token prediction, they are also masked by blocking entries with $j > i$, so a position cannot use later tokens. For Unit 1, the important facts are: rows of A_{att} are weights, the weights add to 1, and $A_{\text{att}}V$ forms weighted averages of the rows of V.

For the phrase "small red bird", masking makes the attention-weight matrix lower triangular, as in the figure. The "small" position can use only itself, so its row is $(1, 0, 0)$. The "bird" position comes last, so it can use all three tokens, as in Gathering context for the bird position (U1-LO1, U1-LO8).

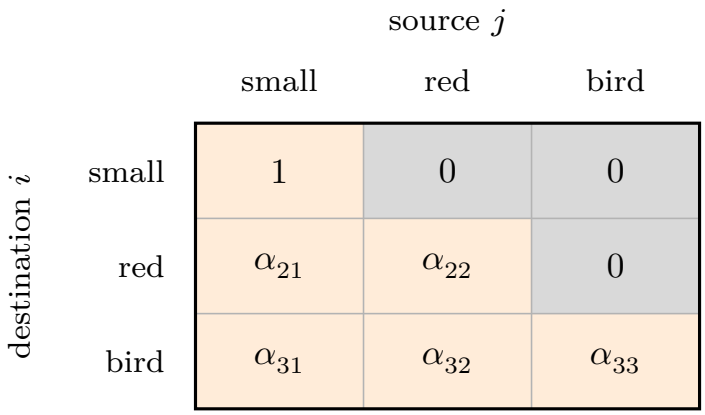


Figure. The masked attention-weight matrix for "small red bird". Gray entries, with $j > i$, are blocked and equal 0. Orange entries, with $j \leq i$, are allowed. Each row still adds to 1.

Unit 3: Softmax and nonlinear attention

Activity: A one-query softmax weighting map (U3-LO7, U1-LO8)

Recall the attention calculation in Activity Gathering context for the bird position (U1-LO1, U1-LO8). We computed query-key dot-product scores, converted them into weights, and formed a weighted average of the value vectors. There, the scores were positive, and we obtained each weight by dividing its score by the sum of the scores:

$$\alpha_i = \frac{s_i}{s_1 + s_2 + s_3}, \qquad i = 1, 2, 3.$$

Here we use the same sequence of operations but replace division by the sum of the scores with softmax. We will view the resulting attention calculation as a nonlinear vector-valued map.
Let

$$K = \begin{bmatrix} 3 & 0 \\ 0 & 3 \\ -3 & -3 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

For a query vector

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix},$$

define the score vector

$$\mathbf{s} = K\mathbf{q}.$$

The softmax rule first exponentiates each score and then divides by the sum of these exponentials:

$$\alpha_i(\mathbf{q}) = \frac{e^{s_i}}{e^{s_1} + e^{s_2} + e^{s_3}}, \qquad i = 1, 2, 3.$$

We write the resulting weight vector as

Activity: A one-query softmax weighting map (U3-LO7, U1-LO8) (continued)

$$\alpha(\mathbf{q}) = \mathrm{softmax}(\mathbf{s}).$$

Because each exponential is positive, these weights are positive and add to 1, even when the original scores are zero or negative. The output is still a weighted average of the value vectors:

$$F(\mathbf{q}) = V^T \alpha(\mathbf{q}).$$

Since $V^T\alpha = (\alpha^T V)^T$, this is the same weighted average of the rows of V as in Example The attention activity in matrix form, written as a column vector.

1. Compute $\mathbf{s} = K\mathbf{q}$ in terms of q_1 and q_2.
2. What are the rows of V?
3. Explain why $F(\mathbf{q})$ is a weighted average of the rows of V.
4. Compute $\alpha(\mathbf{0})$ and $F(\mathbf{0})$.
5. Explain why F is not a linear map.
6. What operation makes F nonlinear?

Note: Softmax outputs stay in the value triangle

Changing $\mathbf{q}$ changes the score vector, and softmax adjusts the three weights together, but they stay positive and add to 1. Therefore, the output always lies strictly inside the triangle with vertices $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{v}_3$.

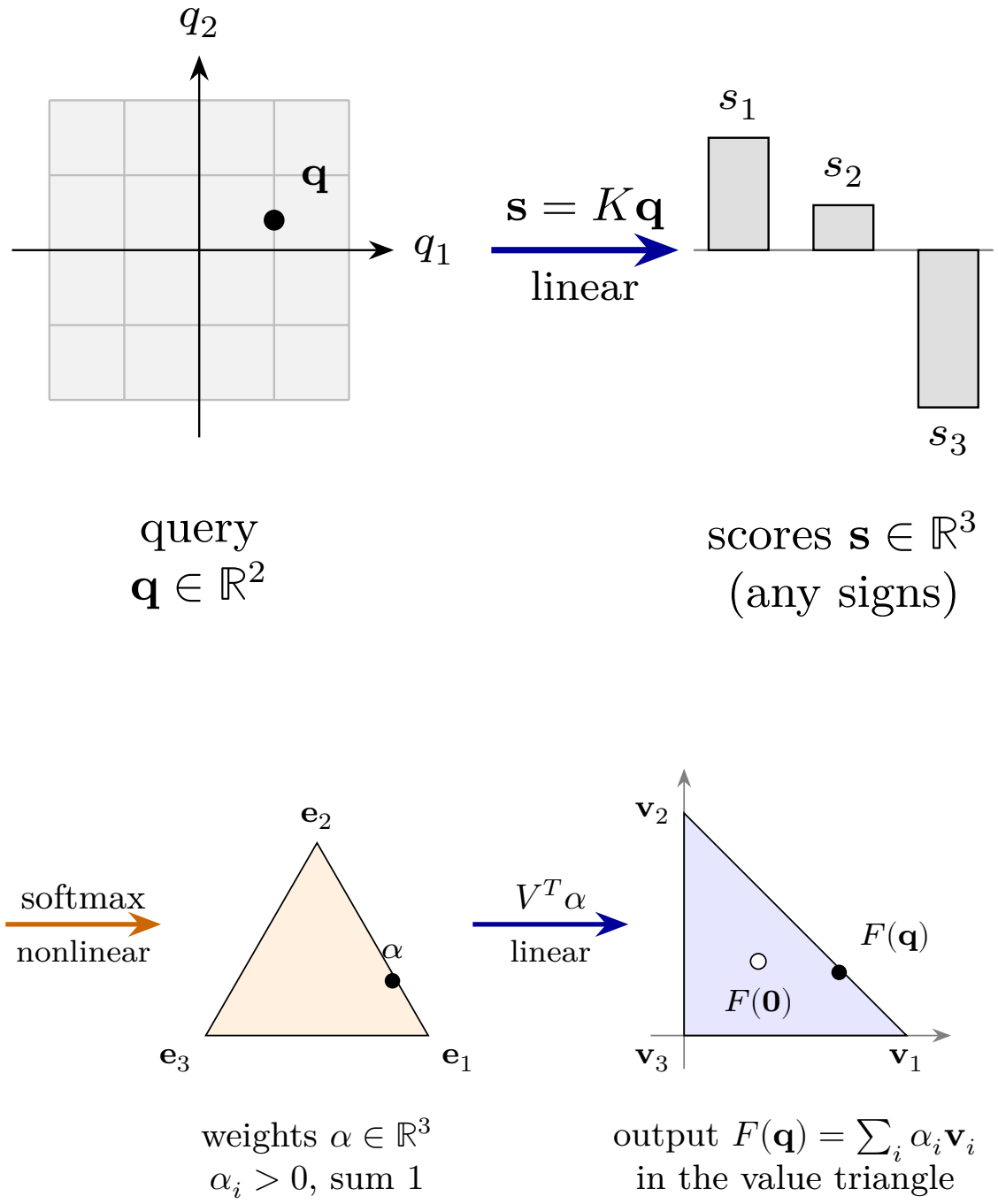


Figure. Query to scores in the upper row; softmax weights and output in the lower row.

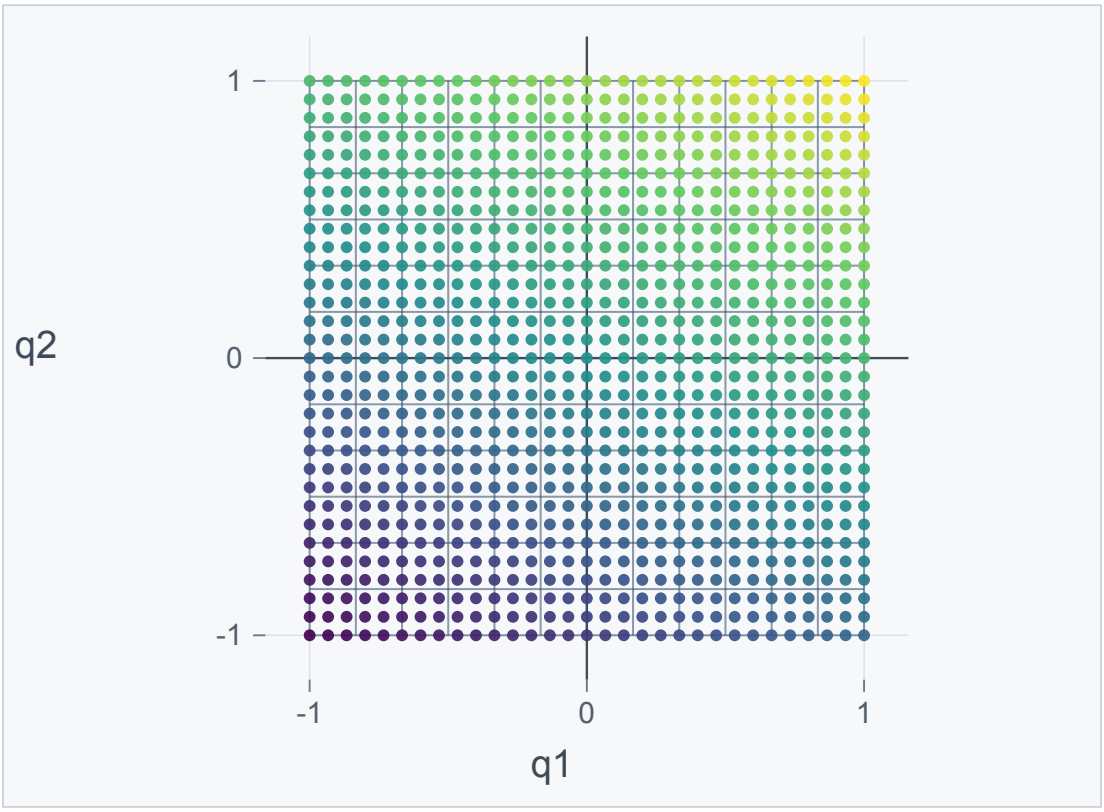


Figure. Input grid. The corresponding output grid follows on the next page.

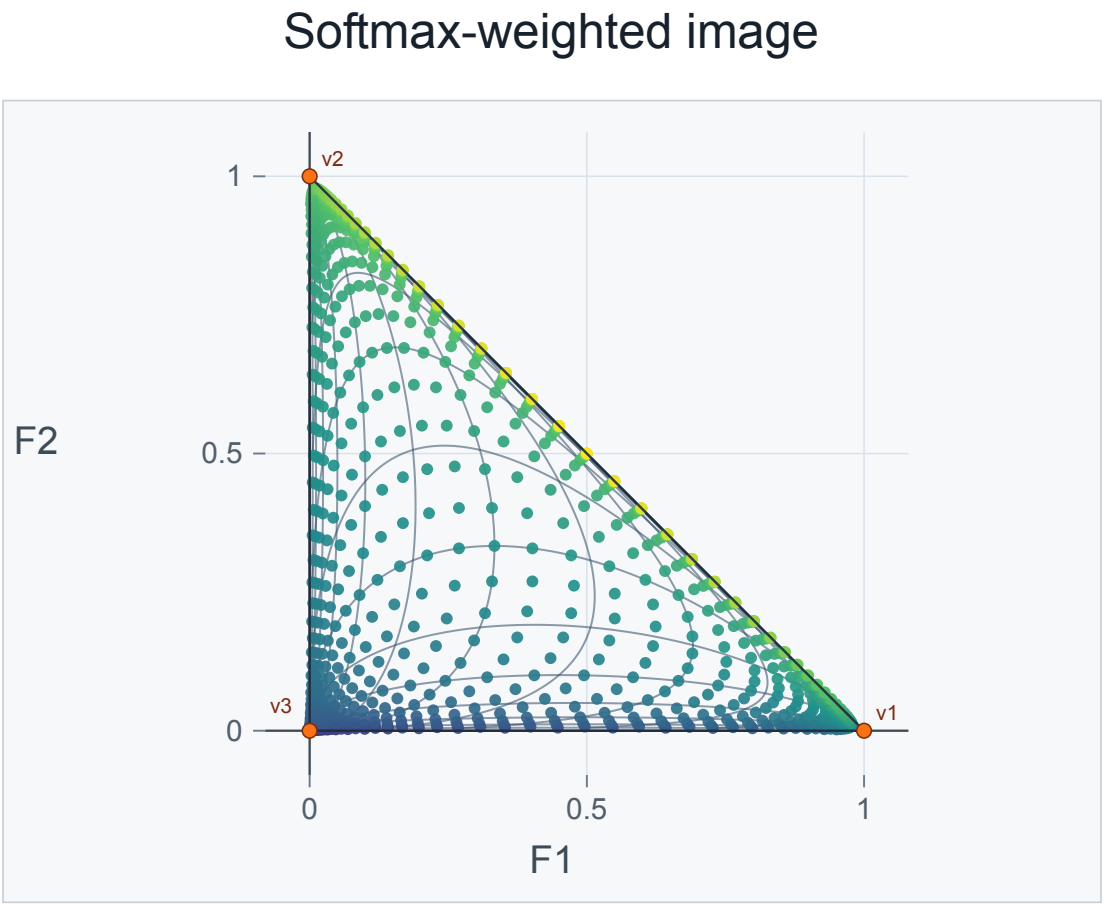


Figure. Square-grid visualization for an attention-style softmax map.

Note: An attention-style nonlinear block

In Activities Gathering context for the bird position (U1-LO1, U1-LO8) and Many-token attention output (U1-LO3, U1-LO6, U1-LO8), we treated attention as three visible operations:

$$\text{dot-product scores} \longrightarrow \text{weights} \longrightarrow \text{weighted averages.}$$

For a sequence of token vectors stored as rows of a matrix X, fixed weight matrices produce

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

The score matrix is

$$S = QK^T.$$

The entry S_{ij} compares the query for token i with the key for token j. A weighting rule, such as row-wise softmax, converts each row of S into weights. Write the resulting attention-weight matrix as A_{att}. The attention output is

$$H = A_{\text{att}}V.$$

If A_{att} were fixed, then $V \mapsto A_{\text{att}}V$ would be a linear map: each row of H would be a weighted average of rows of V. In attention, A_{att} is not fixed. It is computed from X through Q, K, scores, and softmax. Thus the full rule $X \mapsto H$ is generally nonlinear.

The Unit 3 question is local. Near one fixed input X_0, a nonlinear rule can be approximated by a linear map. After the entries of X and H are arranged as vectors, that local linear map is represented by a Jacobian matrix.

Activity: What depends on the input? (U3-LO5, U3-LO7)

For a score matrix $S = [S_{ij}] \in \mathbb{R}^{L\times L}$, row-wise softmax applies the softmax rule separately to each row:

$$\left[\text{softmax}_{\text{rows}}(S)\right]_{ij} = \frac{e^{S_{ij}}}{\sum_{\ell=1}^{L} e^{S_{i\ell}}}, \qquad i, j = 1, \dots, L.$$

The denominator uses only scores from row i, so each row of the resulting matrix contains positive weights that add to 1.

Assume the weight matrices W_Q, W_K, and W_V are fixed. The input is the token matrix X. Define

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$S = QK^T, \qquad A_{\text{att}} = \text{softmax}_{\text{rows}}(S), \qquad H = A_{\text{att}}V.$$

Activity: What depends on the input? (U3-LO5, U3-LO7) (continued)

1. Which objects are fixed before the input X is chosen?
2. Which objects are computed from X?

Activity: Comparing square-grid visualizations (U3-LO1, U3-LO5, U3-LO7, U1-LO8)

This unit used three square-grid visualizations: the nonlinear shear from Corners are not enough (U3-LO1), the sigmoid block from A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7), and the attention-style softmax map from A one-query softmax weighting map (U3-LO7, U1-LO8).

Map A:

$$F\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} x + y^2 \\ y \end{bmatrix}.$$

Map B: a two-output sigmoid block

$$\mathbf{s} = W_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{u} = \sigma(\mathbf{s}), \qquad N(\mathbf{x}) = W_2\mathbf{u} + \mathbf{b}_2.$$

Map C: an attention-style softmax map

$$F_{\text{att}}(\mathbf{q}) = V^T \operatorname{softmax}(K\mathbf{q}).$$

1. Which map agrees with a linear shear on the four corners but not at an edge midpoint?

Activity: Comparing square-grid visualizations (U3-LO1, U3-LO5, U3-LO7, U1-LO8) (continued)

2. Which map is built from affine maps with a coordinatewise sigmoid between them?
3. For which map is the local matrix computed by $W_2 D_{\mathrm{a}} W_1$?
4. Which map produces outputs strictly inside the triangle with vertices $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{v}_3$?
5. For which map does the nonlinear step come from softmax weights that depend on the input?
6. In all three examples, what does the square-grid visualization show that four corners alone may miss?

